Mochi - Security Analysis

1. Introduction

Mochi is split into three parts: a Vue 3 frontend, a Java REST API built with JAX-RS, and a PostgreSQL database running on `bronto.ewi.utwente.nl`. Most of the security-sensitive logic sits in the API layer, since that is where login, registration and all database access happen. Below we go through the threats we considered, what we did about them in the code, and where we know the system is still weak.

2. Threats and What We Did About Them

Threat	How Mochi handles it
SQL Injection	Every query that uses user input goes through a PreparedStatement with bound parameters. We never build SQL by gluing strings
Stolen / guessed passwords	Passwords are hashed with BCrypt (work factor 12). The plain password is never written to the database.
Weak passwords	Registration refuses anything shorter than 8 characters or missing a digit and a capital letter, and it checks this on the server, not just in the browser.
Stolen session	Sessions use a token that expires after 30 minutes, so a leaked token is only useful for a short window.
Bad / malformed input	Values are checked and converted to the right type on the server before they touch the query. Anything that does not fit is rejected.
Inconsistent data	Registration runs its read-then-insert inside a transaction (autoCommit off) so two people signing up at the same time cannot clash.

3. The Mechanisms in More Detail

3.1 Prepared Statements (SQL Injection)

Anywhere user input reaches the database, we use a **PreparedStatement**. Login is a good example: we run `SELECT password FROM person WHERE username = ?` and fill the placeholder with `statement.setString(1, ...)`. Because the query shape is fixed and the value is sent separately, the database treats the input as data, not as code. A classic payload like `' OR '1'='1` just becomes a username that does not exist instead of changing the query.

3.2 Why setAutoCommit(false)

In registration we turn off auto-commit with `conn.setAutoCommit(false)` so that counting the existing users and inserting the new one happen as one transaction. Without this, two registrations running at the same moment could both read the same user count and try to use the same `user_id`. Wrapping them together avoids that race and stops **phantom** reads and similar anomalies, so the database always ends up in a clean state rather than half-written.

3.3 BCrypt for Passwords

We never store passwords as plain text. On sign-up the password is hashed with `BCrypt.hashpw(password, BCrypt.gensalt(12))`. BCrypt adds a random salt for each user and is intentionally slow, which makes brute-force attempts and precomputed rainbow tables

impractical. At login we compare with `BCrypt.checkpw`, so even someone with full read access to the database cannot get the original passwords back.

3.4 Token Sessions (30 minutes)

Once a user logs in they get a session token that **expires after 30 minutes**. Keeping the lifetime short means that if a token leaks, the attacker only has a small window before it stops working and the user has to log in again. This also lets the API stay stateless instead of holding server-side session state.

3.5 Input Validation and Password Rules

We validate on the server before anything reaches the database. For passwords specifically:

- Cannot be null or empty.
- At least 8 characters long.
- Must contain at least one digit and one uppercase letter.

Other inputs are converted to a concrete type as soon as they arrive — dates through `Date.valueOf(...)` and IDs through `Integer.parseInt(...)` — so malformed values fail right away and never reach the SQL.

4. Limitations and Assumptions

- **XSS is only partly covered.** We lean on Vue's automatic escaping and have not added a Content-Security-Policy header, so output handling could be hardened further.
- **HTTPS is assumed.** We assume the API runs over HTTPS in production so tokens and passwords are not sent in clear text.
- **Authorization needs a second pass.** Login works, but we still need to confirm every endpoint checks that a user can only see and edit their own data.